

CSCE 40103: INTRO TO AI FOR SECURITY

LAB 2: PHISHING URL CLASSIFICATION WITH DATA CLEANING, FEATURE PREPARATION, CROSS-VALIDATION, AND TUNING

Introduction

In this lab, you will use the **PhiUSIIL Phishing URL Dataset** to build a simple ML workflow. You will load the dataset, inspect it, clean selected values, create simple engineered features, split the data, scale numeric values, train baseline models, use stratified k-fold cross-validation, tune a Decision Tree, and evaluate the results.

Dataset

- Dataset file: PhiUSIIL_Phishing_URL_Dataset.csv
- Target column: label
- Class 0: Legitimate URL
- Class 1: Phishing URL

Instructions

- Create a Jupyter Notebook named `Lab1-LastName.ipynb`.
- Place `PhiUSIIL_Phishing_URL_Dataset.csv` in the same folder as your notebook.
- Use markdown headings for each lab step.
- Run every code cell before submitting.
- Include your name, date, course name, and lab number at the top of the notebook.

Learning Objectives

- Load and inspect a real cybersecurity dataset.
- Check missing values, duplicate rows, and class distribution.
- Create simple engineered features from existing columns.
- Prepare selected numeric features for machine learning.
- Train Logistic Regression and Decision Tree classifiers.
- Evaluate models using accuracy, precision, recall, F1-score, and confusion matrix.
- Use stratified k-fold cross-validation.
- Tune Decision Tree hyperparameters using *GridSearchCV*.

Required Packages

Use the following imports at the beginning of your notebook.

```
import pandas as pd
import numpy as np

import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.model_selection import StratifiedKFold
from sklearn.model_selection import cross_validate
from sklearn.model_selection import GridSearchCV

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
```

Function and Method Parameters Used in This Lab

Function or Method	Important Parameters	Purpose
pd.read_csv()	filepath_or_buffer	Loads the CSV file.
df.head()	n=5	Displays the first five rows.
df.info()	No required parameter	Displays column names, data types, and non-null counts.
df.isnull().sum()	No required parameter	Counts missing values by column.
df.duplicated().sum()	No required parameter	Counts duplicate rows.
sns.countplot()	data, x	Creates a count plot for the target labels.
sns.histplot()	data, x, bins, kde	Creates a histogram for a numeric feature.
np.log1p()	x	Computes $\log(1 + x)$, which works safely with zero values.
sns.heatmap()	data, annot, cmap	Creates a correlation heatmap.
train_test_split()	X, y, test_size, stratify, random_state	Creates training and testing sets.
StandardScaler()	No required parameter	Standardizes numeric features.
scaler.fit_transform()	X_train	Fits the scaler on training data and transforms the training data.
scaler.transform()	X_test	Transforms test data using the training-fitted scaler.
LogisticRegression()	max_iter, class_weight, random_state	Creates a Logistic Regression classifier.
DecisionTreeClassifier()	max_depth, min_samples_split, min_samples_leaf, class_weight, random_state	Creates a Decision Tree classifier.
StratifiedKFold()	n_splits, shuffle, random_state	Creates stratified folds for cross-validation.
cross_validate()	estimator, X, y, cv, scoring	Evaluates a model across folds using multiple metrics.
GridSearchCV()	estimator, param_grid, cv, scoring	Finds the best hyperparameter combination.
classification_report()	y_true, y_pred, zero_division	Displays precision, recall, F1-score, and support.
confusion_matrix()	y_true, y_pred	Computes confusion matrix values.
ConfusionMatrixDisplay()	confusion_matrix, display_labels	Displays a confusion matrix figure.

Step 1: Create the Notebook

Create a new Jupyter Notebook named:

Lab1-LastName.ipynb

At the top of the notebook, add a markdown cell with your name, date, course name, and lab number.

Step 2: Load the Dataset

Load the PhiUSIIL dataset into a DataFrame named `df`.

```
df = pd.read_csv("PhiUSIIL_Phishing_URL_Dataset.csv")  
df.head()
```

Required output: First five rows of the dataset.

Step 3: Inspect the Dataset

Display the dataset size, column names, and data types.

```
print("Rows and columns:", df.shape)
```

```
print("Column names:")
```

```
print(df.columns)
```

```
print("Dataset information:")
```

```
df.info()
```

Required output: dataset shape, column names, and data type summary.

Step 4: Check Missing Values and Duplicate Rows

Check whether the dataset contains missing values and duplicate rows.

```
missing_values = df.isnull().sum()
print(missing_values[missing_values > 0])

duplicate_count = df.duplicated().sum()
print("Number of duplicate rows:", duplicate_count)
```

Required output: missing value summary and duplicate row count.

Step 5: Inspect the Target Variable

Display class counts and percentages, then create a class distribution plot.

```
class_counts = df["label"].value_counts()
class_percentages = df["label"].value_counts(normalize=True) * 100

print("Class counts:")
print(class_counts)

print("Class percentages:")
print(class_percentages.round(2))

sns.countplot(data=df, x="label")
plt.title("Class Distribution")
plt.xlabel("Class: 0 = Legitimate, 1 = Phishing")
plt.ylabel("Number of Records")
plt.show()
```

Required output: class counts, class percentages, and class distribution plot.

Step 6: Select Useful Features

Use the provided candidate list and keep only the columns that exist in the dataset.

```
candidate_features = [  
    "URLLength",  
    "DomainLength",  
    "NoOfSubDomain",  
    "NoOfLettersInURL",  
    "NoOfDegitsInURL",  
    "NoOfOtherSpecialCharsInURL",  
    "IsHTTPS",  
    "NoOfExternalRef",  
    "LineOfCode",  
    "LargestLineLength"  
]  
  
selected_features = []  
  
for col in candidate_features:  
    if col in df.columns:  
        selected_features.append(col)  
  
print("Selected features:")  
print(selected_features)
```

Required output: selected feature list.

Step 7: Explore Numeric Feature Distributions

Create histograms for important numeric features.

```
features_to_plot = ["URLLength", "DomainLength", "NoOfExternalRef"]

for feature in features_to_plot:
    if feature in df.columns:
        sns.histplot(data=df, x=feature, bins=40, kde=True)
        plt.title(feature + " Distribution")
        plt.xlabel(feature)
        plt.ylabel("Number of Records")
        plt.show()
```

Required output: histogram figures.

Step 8: Create Simple Engineered Features

Create log-transformed versions of skewed numeric features.

```
work_df = df.copy()

if "URLLength" in work_df.columns:
    work_df["log_URLLength"] = np.log1p(work_df["URLLength"])

if "NoOfExternalRef" in work_df.columns:
    work_df["log_NoOfExternalRef"] = np.log1p(work_df["NoOfExternalRef"])

work_df.head()
```

Required output: DataFrame preview showing the new engineered columns.

Step 9: Compare Raw and Log-Transformed Values

Create raw and log-transformed histograms for URLLength.

```
if "URLLength" in work_df.columns:  
    sns.histplot(data=work_df, x="URLLength", bins=40, kde=True)  
    plt.title("Raw URLLength")  
    plt.show()  
  
    sns.histplot(data=work_df, x="log_URLLength", bins=40, kde=True)  
    plt.title("Log-Transformed URLLength")  
    plt.show()
```

Required output: raw URLLength histogram and log-transformed URLLength histogram.

Step 10: Update the Feature List

Add the engineered features to the selected feature list.

```
final_features = selected_features.copy()

if "log_URLLength" in work_df.columns:
    final_features.append("log_URLLength")

if "log_NoOfExternalRef" in work_df.columns:
    final_features.append("log_NoOfExternalRef")

print("Final features:")
print(final_features)
```

Required output: final feature list.

Step 11: Correlation Analysis

Create a correlation heatmap using the final numeric features and the label.

```
corr = work_df[final_features + ["label"]].corr()

plt.figure(figsize=(10, 8))
sns.heatmap(corr, annot=False, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```

Required output: correlation heatmap.

Step 12: Create X and y

Create the feature matrix X and target vector y.

```
X = work_df[final_features]
y = work_df["label"]

print("X shape:", X.shape)
print("y shape:", y.shape)
```

Required output: shapes of X and y.

Step 13: Train-Test Split

Split the data into training and testing sets. Use `stratify=y` to preserve class proportions.

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    stratify=y,
    random_state=40103
)

print("Training rows:", X_train.shape[0])
print("Testing rows:", X_test.shape[0])

print("Training class distribution:")
print(y_train.value_counts(normalize=True).round(3))

print("Testing class distribution:")
print(y_test.value_counts(normalize=True).round(3))
```

Required output: training and testing row counts plus class proportions.

Step 14: Fill Missing Values

Use the training median to fill missing values. Use the same training median for the test set.

```
train_medians = X_train.median()

X_train_clean = X_train.fillna(train_medians)
X_test_clean = X_test.fillna(train_medians)

print("Missing values in X_train_clean:", X_train_clean.isnull().sum().sum())
print("Missing values in X_test_clean:", X_test_clean.isnull().sum().sum())
```

Required output: missing value counts after filling.

Step 15: Scale Numeric Features

Scale the features using StandardScaler. Fit the scaler only on the training data.

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train_clean)
X_test_scaled = scaler.transform(X_test_clean)

print("Scaled training shape:", X_train_scaled.shape)
print("Scaled testing shape:", X_test_scaled.shape)
```

Required output: scaled training and testing shapes.

Step 16: Train Logistic Regression

Create, train, and evaluate a Logistic Regression model.

```
log_model = LogisticRegression(  
    max_iter=1000,  
    class_weight="balanced",  
    random_state=40103  
)  
  
log_model.fit(X_train_scaled, y_train)  
log_pred = log_model.predict(X_test_scaled)  
  
print(classification_report(y_test, log_pred, zero_division=0))  
  
log_cm = confusion_matrix(y_test, log_pred)  
disp = ConfusionMatrixDisplay(  
    confusion_matrix=log_cm,  
    display_labels=["Legitimate", "Phishing"]  
)  
disp.plot(values_format="d")  
plt.title("Logistic Regression Confusion Matrix")  
plt.show()
```

Required outputs: classification report and confusion matrix.

Step 17: Train Decision Tree

Create, train, and evaluate a Decision Tree model.

```
tree_model = DecisionTreeClassifier(  
    random_state=40103,  
    class_weight="balanced"  
)  
  
tree_model.fit(X_train_clean, y_train)  
tree_pred = tree_model.predict(X_test_clean)  
  
print(classification_report(y_test, tree_pred, zero_division=0))  
  
tree_cm = confusion_matrix(y_test, tree_pred)  
disp = ConfusionMatrixDisplay(  
    confusion_matrix=tree_cm,  
    display_labels=["Legitimate", "Phishing"]  
)  
disp.plot(values_format="d")  
plt.title("Decision Tree Confusion Matrix")  
plt.show()
```

Required outputs: classification report and confusion matrix. Decision Trees do not require scaling, so use `X_train_clean` and `X_test_clean`.

Step 18: Create a Metric Summary Table

Create a function to compute model metrics and display a comparison table.

`get_metrics()`

```
def get_metrics(model_name, y_true, y_pred):  
    return {  
        "Model": model_name,  
        "Accuracy": accuracy_score(y_true, y_pred),  
        "Precision": precision_score(y_true, y_pred, zero_division=0),  
        "Recall": recall_score(y_true, y_pred, zero_division=0),  
        "F1": f1_score(y_true, y_pred, zero_division=0)  
    }  
  
results = [  
    get_metrics("Logistic Regression", y_test, log_pred),  
    get_metrics("Decision Tree", y_test, tree_pred)  
]  
  
results_df = pd.DataFrame(results)  
results_df
```

Required output: model comparison table.

Step 19: Stratified k-Fold Cross-Validation

Use stratified 5-fold cross-validation to compare the two models more reliably.

```
cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=40103
)

scoring = ["accuracy", "precision", "recall", "f1"]

X_clean = X.fillna(X.median())
X_scaled = scaler.fit_transform(X_clean)

log_cv = cross_validate(
    log_model,
    X_scaled,
    y,
    cv=cv,
    scoring=scoring
)

tree_cv = cross_validate(
    tree_model,
    X_clean,
    y,
    cv=cv,
    scoring=scoring
)

cv_summary = pd.DataFrame([
    {
        "Model": "Logistic Regression",
        "Accuracy": log_cv["test_accuracy"].mean(),
        "Precision": log_cv["test_precision"].mean(),
        "Recall": log_cv["test_recall"].mean(),
        "F1": log_cv["test_f1"].mean()
    },
    {
        "Model": "Decision Tree",
        "Accuracy": tree_cv["test_accuracy"].mean(),
        "Precision": tree_cv["test_precision"].mean(),
        "Recall": tree_cv["test_recall"].mean(),
        "F1": tree_cv["test_f1"].mean()
    }
])

cv_summary
```

Required output: cross-validation summary table.

Step 20: Hyperparameter Tuning for Decision Tree

Tune only the Decision Tree model. Random Forest is not used in this lab.

```
param_grid = {
    "max_depth": [3, 5, 10, None],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4]
}

tree_grid = GridSearchCV(
    estimator=DecisionTreeClassifier(
        random_state=40103,
        class_weight="balanced"
    ),
    param_grid=param_grid,
    cv=cv,
    scoring="f1"
)

tree_grid.fit(X_train_clean, y_train)

print("Best parameters:")
print(tree_grid.best_params_)

print("Best cross-validation F1:")
print(tree_grid.best_score_)
```

Required outputs: `best_params_` and `best_score_`.

Step 21: Evaluate the Tuned Decision Tree

Use the best Decision Tree model to predict the test set.

```
best_tree = tree_grid.best_estimator_  
  
tuned_tree_pred = best_tree.predict(X_test_clean)  
  
print(classification_report(y_test, tuned_tree_pred, zero_division=0))  
  
tuned_cm = confusion_matrix(y_test, tuned_tree_pred)  
disp = ConfusionMatrixDisplay(  
    confusion_matrix=tuned_cm,  
    display_labels=["Legitimate", "Phishing"]  
)  
disp.plot(values_format="d")  
plt.title("Tuned Decision Tree Confusion Matrix")  
plt.show()
```

Required outputs: classification report and confusion matrix.

Step 22: Final Comparison Table

Create a final table comparing all three model versions.

```
final_results = [  
    get_metrics("Logistic Regression", y_test, log_pred),  
    get_metrics("Decision Tree", y_test, tree_pred),  
    get_metrics("Tuned Decision Tree", y_test, tuned_tree_pred)  
]  
  
final_results_df = pd.DataFrame(final_results)  
final_results_df
```

Required output: final comparison table.

What to Submit

- Notebook file: Lab1-LastName.ipynb